

Plant calendar — Function design

006_DD-FN version 1 — SCR-006, FN-037–040, REQ-070–075. WI-010 approved revision 1 step 8. Backend design only; not implemented.

1. Document control and references

Field	Value
Document ID / version	006_DD-FN / 1
System / subsystem	ProductionManagementAI / Master data
Work item / author	WI-010 / Agent
Created / updated	2026-10-02 / 2026-10-02
State	Submitted for review, not implemented

Version	Date	Author	Change
1	2026-10-02	Agent	Initial backend methods, exact resolver/capacity and transaction outcome design

Inputs: [main DD](#), [API](#), [DB](#), [BD](#), [requirements](#) and [plan](#), all preceding designs approved. API owns HTTP fields/errors; main DD owns UI state; this file owns backend methods. DD-SPD is still reserved, not authored. Existing order/master algorithms unchanged.

2. Module and method index

Planned PlantCalendarService in Application uses a typed calendar repository port; Infrastructure implements EF/Npgsql projections/transactions. Resolver/capacity are pure testable rules; controller contains no SQL or scheduling logic. Names are proposals, not compiled files. All methods Agent, created/updated 2026-10-02.

No.	Method	Caller / requirement function	Return
M-01	MonthAsync	API-PC-01 / FN-037	<code>CalendarResult<MonthResult></code>
M-02	DayAsync	API-PC-02 / FN-037/039	<code>CalendarResult<DayResult></code>
M-03	WeeklyAsync	API-PC-03 / FN-038	<code>CalendarResult<WeeklyPage></code>

No.	Method	Caller / requirement function	Return
M-04	SaveWeeklyAsync	API-PC-04 / FN-038	CalendarResult<MutationResult>
M-05	WithdrawWeeklyAsync	API-PC-05 / FN-038	CalendarResult<MutationResult>
M-06	SaveExceptionAsync	API-PC-06 / FN-039	CalendarResult<MutationResult>
M-07	RemoveExceptionAsync	API-PC-07 / FN-039	CalendarResult<MutationResult>
M-08	LineChoicesAsync	API-PC-08 / FN-037/040	CalendarResult<LineChoicePage>
M-09	ProductChoicesAsync	API-PC-09 / FN-040	CalendarResult<ProductChoicePage>
M-10	CapacityAsync	API-PC-10 / FN-040	CalendarResult<CapacityResult>
M-11	ExceptionHistoryAsync	API-PC-11 / FN-039	CalendarResult<ExceptionPage>
M-12	ResolveDay	M-01/02/10	DaySummary
M-13	FloorCapacity	M-10	Canonical quantity string
M-14	ReadSnapshotAsync	All read methods	Typed projected result or failure
M-15	WriteTransitionAsync	M-04/05/06/07	MutationResult or certainty-tagged failure

Shared utilities, arguments and value mapping

Utility	Use / boundary
IPlantClock + TimeProvider	Existing clock DateOf(utc)/TimeZoned and captured UTC instant; no browser/SQL CURRENT_DATE basis
AppDbContext / EF Core / Npgsql	Existing persistence stack; no additional ORM or provider
Calendar input validators	Revalidate typed invariants and normalization per approved API; controller strict parser does not replace business checks
Existing Identity policy	Admin/Operator before target lookup; no separate PostgreSQL role per user
Invariant formatter	Canonical UUID/version/decimal/date strings; no binary float or locale-dependent transport

Public methods take their typed query/command corresponding to the named API and CancellationToken. Query objects contain only allowed API fields after strict parsing; commands include observed long revision and nullable target UUID. No arbitrary client SQL, is_current, commit_revision or replacement revision. Pure helper arguments are listed in sections 4/5; no I/O/cancellation side effects. Request data to external services: none; no holiday provider or outgoing HTTP. Response field catalogs are exclusively API-owned; mapping here is procedural: weekday mask -> ordered weekday strings, current snapshots -> exact API schema, null markers -> fallback absence, exact integers -> canonical decimal strings, typed failure -> approved code/status/writeOutcome. No new response properties.

3. ReadSnapshotAsync — coherent read boundary (M-14)

Arguments: typed query, projection delegate, CancellationToken. Return: typed API projection or CalendarFailure. Owns one read transaction, not a long-lived cache.

Step	Processing	Calls
1	Link request cancellation to 20s feature deadline; set command timeout 15s	Scoped repository settings
2	Begin read-only REPEATABLE READ; apply SET LOCAL statement_timeout=15000ms and lock_timeout=5000ms	EF transaction, parameterized setting calls
3	Read state id=1, then capture utc=TimeProvider.GetUtcNow(), plantToday=clock.DateOf(utc) once; validate stored/configured timezone equality	State projection, IPlantClock
4	If snapshotVersion supplied, compare against state revision; page 2+ already requires it; mismatch CALENDAR_STALE	Calendar validators
5	Read requested references/rules/count/page in this same snapshot; project only required columns	Named read method below
6	Finish read transaction before returning typed result; dispose and restore scoped command settings on all paths	Repository wrapper

State absent is allowed for reads: context activation/version null, no invented initial weekdays. Timezone mismatch is 503 as API, no data reinterpretation. Capture date only once per read; midnight during later queries does not mix date bases. Snapshot starts at first SQL read; all projections share it. This follows [PostgreSQL isolation](#). READ COMMITTED writes use a different boundary. No tracking cache spanning requests, no lazy loading/N+1. Counts and items are read before transaction ends.

4. ResolveDay — precedence and hours (M-12)

Arguments: requested DateOnly, CalendarContext, selected line or null, latest plant/line exception snapshots for that date, latest applicable weekly snapshot. Return DaySummary; no persisted capacity or historical master-hours claim.

Step	Processing	Calls
1	Before activatedOn or missing state: Unavailable, hours null, source None, fallback [], editable=false	Context guard
2	Build candidates: current nonremoved line exception, current nonremoved plant exception, applicable current nonwithdrawn weekly	Snapshot projection
3	Pick first candidate; no candidate means Unavailable. Remaining candidates become fallback in priority order (max 2)	Source mapper
4	Working exception uses its state; weekly uses Monday-based weekday mask. Closed gives hours "0"/Closed basis	Weekday mapping
5	Working explicit exception uses exact hours/Explicit; otherwise selected line current hours/LineCurrent; no selected line gives null/LineDependent	Existing line projection

Step	Processing	Calls
6	editable for save/create = covered, date>=plantToday and plant scope or active line; preserve source IDs/reason	API mapper

Never revive an older row of the same logical target when its current row is a removal/withdrawal marker. Weekly lookup uses is_current AND NOT is_withdrawn, effective_from<=date, greatest effective_from. A withdrawn later start yields its predecessor date, not an older revision of the withdrawn start. Weekday bit index = ((int)date.DayOfWeek + 6) % 7; Monday bit 0, Sunday bit 6; working iff (mask & (1 << index)) != 0. All-closed mask 0 is valid. Line exception wins wholly: inherited hours use line baseline, even if lower plant exception had explicit hours. No partial merge. Past inherited hours are labeled current-setting, never past capacity. A retired line's editable=false for save/create does not prohibit allowed existing today/future removal; removal eligibility independently checks captured exception/date per M-07.

5. FloorCapacity — exact conservative arithmetic (M-13)

Arguments: valid effective decimal hours H (0..24, scale<=3), valid decimal minutes/unit T (>0, <=999999999.999, scale<=3), approved product unit. Return invariant quantity string. Invalid persisted values are invariant failures, not silently clamped; the caller checks eligibility before arithmetic.

Convert exactly to integers $h=H1000$ and $t=T1000$ without rounding. Both are whole numbers because storage/input scale was validated. $N=\text{checked}(h*60)$.

Unit	Calculation	Return
個 / 本 / 枚 / 台 / セット	$q=N \text{ div } t$	Nonnegative integer string
kg / m	$q3=\text{checked}(N*1000) \text{ div } t$	$q3/1000$ as exact decimal, canonical string with trailing zeros removed

Integer div truncates nonnegative operands, exactly implementing floor. Do not first divide .NET decimal then floor: rounded intermediate division can cross a boundary. $h\leq 24000$, $N\leq 1440000$, $N*1000\leq 1440000000$; $t\leq 999999999999$. All fit Int64; use checked conversion/multiplication, never double/float. Quantity upper bound is 1440000 when 24 hours and 0.001 minute/unit. Closed valid pair computes zero.

Hours	Minutes/unit	Unit	Expected quantity / purpose
4	0.125	個	"1920", reference reopening example
0.001	0.007	個	"8", floor a nonterminating ratio
0.001	0.007	kg	"8.571", exact three-place floor
1	60.001	個	"0", below integer boundary
0.001	60	m	"0.001", exact smallest quantum

Hours	Minutes/unit	Unit	Expected quantity / purpose
0.001	60.001	m	"0", just below quantum; never round upward
24	0.001	kg	"1440000", maximum

These are design vectors; independent arithmetic evaluation is document evidence, not tests of an implemented C# helper. Application test-plan IDs assigned later.

6. WriteTransitionAsync — atomic mutation boundary (M-15)

Arguments: normalized typed command, operation kind (save/withdraw/save exception/ remove), optional line UUID, Cancellation token. Return MutationResult or typed failure with the API's NotApplied/Unknown certainty. No retry loop.

Step	Processing	Calls
1	Apply API deadline/timeouts; begin READ COMMITTED; validate static shapes before locks	Repository transaction
2	For a line exception operation, SELECT existing line by id FOR SHARE; missing is NOT_FOUND. Never lock state first then line	Parameterized line query
3	SELECT state WHERE id=1 FOR UPDATE; absent CALENDAR_NOT_ACTIVATED; compare timezone and observed revision	State query
4	Read current target by exact logical scope/date, including marker; validate submitted target identity and latest id	Operation predicate
5	After all lock waits, capture fresh UTC instant and DateOf(utc); recheck activation, today/future, protected rule and active line (except allowed removal)	Clock, operation rules
6	Compare normalized save payload only after version/target/permission checks. Equal normal payload returns changed=false without revision advance	Semantic comparer
7	Compute checked next=state.revision+1; supersede existing current row by id and is_current=true (must affect 1); INSERT new immutable snapshot/marker with next	Column-scoped update + insert
8	UPDATE state SET revision=next, updated_at_utc=capturedUtc WHERE id=1 AND revision=observed (must affect 1)	State writer
9	Mark CommitIssued immediately before sending commit; return changed=true only after acknowledged CommitConfirmed	Transaction commit

No-op still completes its transaction and obeys known/unknown completion rules; no new snapshot, version or timestamp. New target has no supersede update. Snapshots' created_at_utc and state update use captured UTC instant; date basis uses DateOf of that instant. Clock changes after validation do not change the accepted operation basis. Do not re-read today per statement.

Parameterized locking SQL intent (design only):

```

SELECT id, is_active FROM public.production_lines WHERE id = @lineId FOR SHARE;
SELECT id, activated_on, time_zone_id, revision
FROM public.plant_calendar_state WHERE id = 1 FOR UPDATE;

```

No FOR KEY SHARE substitution: retirement changes a non-key field, and FOR SHARE must conflict with it. Existing order/master lock paths do not acquire calendar. Calendar never locks products/pairs/orders. Lock order line -> state therefore introduces no reverse edge to existing product -> line -> pair order. See [PostgreSQL row locks](#). Hold locks through transaction end; no intermediate SaveChanges transaction/commit. No advisory lock, external call, persisted quantity or calendar-delete trigger.

7. Endpoint-backed method designs

All argument types bind only the approved API fields. All return `CalendarResult<T>` : value or failure, never both. Common parser/auth errors are API-owned. Per-method flows below call M-14/M-15; shared boundaries are not reimplemented independently.

M-01 MonthAsync

Arguments MonthQuery(month, lineId?), ct; return MonthResult.

Step	Processing	Calls
1	Validate real month; use DaysInMonth and day numbers, not final-day AddDays overflow	Input rules
2	Read state, optional retained line, greatest live weekly start before first day and live current starts inside month	ReadSnapshotAsync
3	Batch current exceptions for plant and selected line within first/last date, including markers; dictionary by scope/date	Repository projections
4	Resolve each of 28..31 dates with in-memory predecessor cursor; sort days ascending	ResolveDay

At most 32 live weekly heads and 62 exception heads for a line/month; no history scan or per-day DB query. Typical projected data reads: state, optional line, predecessor weekly, month weekly, batched exceptions (max five); not a runtime performance claim. Source/records remain consistent with one snapshot.

M-02 DayAsync

Arguments DayQuery(date,lineId?), ct; return DayResult.

Step	Processing	Calls
1	Read state, retained line if selected, effective live weekly head and current plant/line heads	ReadSnapshotAsync
2	Resolve day and expose the exact scope/date latest exception including removed marker, or null if never configured	ResolveDay, snapshot mapper
3	Return captured context; past restrictions apply, no capacity computation	API mapper

M-03 WeeklyAsync

Arguments WeeklyQuery(from,to,view,page,snapshotVersion?), ct; return WeeklyPage.

Step	Processing	Calls
1	Enforce inclusive span<=366, page<=10000 and snapshot token rules	Input rules, ReadSnapshotAsync
2	Filter starts inside range; Current uses is_current including withdrawal; History includes retained snapshots	Repository
3	Count and page 20 in effective_from DESC,commit_revision DESC,id ASC; read latest live start strictly before from separately	Repository
4	Map applicableBeforeFrom and current context/version; empty/out-of-page data uses actual totals	API mapper

M-04 SaveWeeklyAsync

Arguments WeeklySave(effectiveFrom,version,targetRevisionId?,workingDays), ct; return MutationResult. Distinct weekday tokens map exactly to mask; [] -> 0.

Step	Processing	Calls
1	Normalize/validate weekdays and exact target date	Input rules
2	Under global lock, null target requires no head; nonnull must identify exact latest same-date snapshot, including withdrawn marker	WriteTransitionAsync
3	Reject past start; today's replacement never changes an older start. Re-add/current correction appends nonwithdrawn snapshot	Weekly operation rules

M-05 WithdrawWeeklyAsync

Arguments WeeklyWithdraw(effectiveFrom,version,targetRevisionId), ct; return MutationResult.

Step	Processing	Calls
1	Under lock, verify current same-date nonwithdrawn target	WriteTransitionAsync
2	Reject activation definition withdrawal and nonfuture start; no-op withdrawal not supported	Weekly operation rules

Step	Processing	Calls
3	Append withdrawn snapshot with mask null; predecessor date becomes applicable on next read	Snapshot writer

M-06 SaveExceptionAsync

Arguments ExceptionSave(version,targetRevisionId?,lineId?,date,isWorking,hours?,reason?), ct; return MutationResult.

Step	Processing	Calls
1	Normalize exact hours/reason and closed/inherited consistency; no implicit zero or rounding	Input rules
2	Lock line then state; new target null only if no head; submitted marker id permits re-add, normal latest id permits same-scope/date edit	WriteTransitionAsync
3	Require active referenced line, covered today/future date; append normal payload with is_removed=false	Exception operation rules

M-07 RemoveExceptionAsync

Arguments ExceptionRemove(version,targetRevisionId,lineId?,date), ct; return MutationResult.

Step	Processing	Calls
1	Lock referenced line (may be retired), then state; current nonremoved exact target mandatory	WriteTransitionAsync
2	Check covered today/future date; absent/removed/mismatched target fails, not false success	Exception operation rules
3	Append removed snapshot with is_working/hours/reason null; preserve old notes and all records	Snapshot writer

M-08 LineChoicesAsync

Arguments ChoiceQuery(q?,page), ct; return LineChoicePage.

Step	Processing	Calls
1	Read context and active line projection; optional q matches code/name literal contains	ReadSnapshotAsync
2	Escape backslash, percent and underscore before parameterized ILIKE with explicit escape; count and page 50 code,id order	Existing query pattern
3	Return current names/hours/state only, not line edit versions; activation absent still allows choices	API mapper

M-09 ProductChoicesAsync

Arguments ProductChoiceQuery(lineld,q?,page), ct; return ProductChoicePage.

Step	Processing	Calls
1	Read required line; unknown 404, retired yields empty with real context	ReadSnapshotAsync
2	Join active pairs/products for line, matching confirmed_unit and confirmed_unit_revision to current product; literal SKU/name filter	Projected repository query
3	Count/page 50 SKU,id order; return current unit generation, never silently reconfirm	API mapper

M-10 CapacityAsync

Arguments CapacityQuery(lineld,productId,date), ct; return CapacityResult.

Step	Processing	Calls
1	Read context and required line/product identities; unknown 404, date<captured today 400	ReadSnapshotAsync
2	Read optional pair plus effective weekly and current plant/line exceptions; keep absent pair distinguishable from missing identities	Repository projections
3	Resolve day; apply unavailable reason priority exactly as API	ResolveDay, eligibility guard
4	If usable, format current coefficient and exact quantity; otherwise quantity/minutes null, no fabricated zero	FloorCapacity, API mapper

Eligibility priority: NotActivated -> LineRetired -> ProductRetired -> PairMissing -> PairRetired -> UnitStale -> CalendarUnavailable. UnitStale if confirmed unit OR confirmed revision differs, including ABA unit changes. Known retired/stale pair stays unavailable even on closed date. Past capacity never computed. A valid closed pair is Available with "0". No mixed units, history recalculation, allocation or order-start gating. Missing pair must not be hidden by an inner join that incorrectly produces NOT_FOUND. Separate projected reads or a safe left join keep required master identity checks explicit.

M-11 ExceptionHistoryAsync

Arguments ExceptionHistoryQuery(date,lineld?,page,snapshotVersion?), ct; return ExceptionPage.

Step	Processing	Calls
1	Read context, scope line if any and snapshot token; retained/past scope is allowed	ReadSnapshotAsync
2	Count/page all exact scope/date revisions including markers, commit_revision DESC,id ASC	Repository
3	Read current head independently of page; return observation, not proof of originating client's commit	API mapper

Nullable line predicate uses explicit plant-null branch or null-safe equality, not SQL line_id = NULL. Page offsets bounded by API and checked arithmetic; large history is paged, never silently truncated.

8. Failure certainty, budgets and telemetry

Maintain operation phases BeforeCommit, CommitIssued, CommitConfirmed and RollbackConfirmed as internal state, not new API response fields. Stage update must precede sending COMMIT. Known validation/target failures before commit roll back before returning NotApplied. Deadline/deadlock/lock failure with acknowledged rollback -> CALENDAR_BUSY/NotApplied. Error during commit without authoritative completion -> CALENDAR_WRITE_UNKNOWN/Unknown; no Retry-After. Generic write 500 remains Unknown per API even when internal diagnostics know more. No success until commit acknowledgement; response delivery loss can still make client Unknown.

No retry execution strategy wraps calendar writes, no savepoint-based replay and no idempotency ledger. Cancellation alone is not rollback proof. Attempt rollback and disposal without reusing an already-cancelled request token, using bounded cleanup <=5s; this is cleanup, not an operation deadline extension for commit. If cleanup cannot establish outcome, retain Unknown and dispose the connection; no request resurrection. An interrupted read is never a partial successful result.

Persistence failure	Mapping / boundary
Named current/commit unique index 23505	Confirm rollback; CALENDAR_TARGET_CHANGED 409; unexpected other unique violation remains UNEXPECTED
Missing target / zero supersede or state update rows	Rollback; typed NOT_FOUND or CALENDAR_STALE/TARGET_CHANGED according to checked condition
Lock/deadlock/cancel before commit	Only known rollback gives BUSY/NotApplied; otherwise safe uncertainty
Persisted range/mask/marker invariant violated	UNEXPECTED, never repair/clamp data or report an invented usable quantity
Checked revision overflow	Roll back, UNEXPECTED; no wrap/reset, owner forward-fix required

Feature deadline 20s, command/statement 15s, lock 5s, client 25s as approved API. Use SET LOCAL settings only inside feature transactions; restore per-context EF command timeout and dispose transactions; pooled connections must not carry calendar settings into old features. All finite operations honor request cancellation. Deadline includes projection/calculation/commit, not just individual commands.

ActivitySource/Meter ProductionManagementAI.PlantCalendar registered in API startup. Top-level span `plant-calendar.<handler>`, handlers exactly the eleven API names. Children plant-calendar.read, plant-calendar.resolve, plant-calendar.write; tags limited to bounded stage/outcome, no raw values. Pure capacity may be measured within resolve; no high-cardinality quantities/IDs/revisions. One final request metric owner (feature-aware middleware/adaptor) records plant_calendar_operations_total and plant_calendar_duration_ms with operation and approved outcome labels. Service supplies outcome but does not count again. 401/403/parser paths also counted once; post-CommitIssued unknown takes priority over cancelled when result cannot be established. Unavailable capacity uses unavailable even with HTTP 200. Logs contain only operation/outcome/traceld, not SQL, dates, search, note, coefficient, token or credentials. No exporter claim.

9. Verification viewpoints and next review

Requirement	Later implementation verification
REQ-070	Exact month bounds/leap days/year 9999; coherent snapshots; empty activation; no N+1
REQ-071	Mask all-closed; latest-head withdrawal/predecessor; today insertion; retained correction
REQ-072	Null scope duplicate; marker re-add; closed/inherit validation; reason Unicode; Cancel no mutation
REQ-073	Whole precedence/reopening; integer-floor boundary vectors; ABA/retirement; invalid vs closed zero
REQ-074	Role gates, unrelated global conflict, lock/midnight races, zero-row rollback, uncertain COMMIT, cleanup and single telemetry accounting
REQ-075	No order/master writes or due-date/window/start algorithm change

No open business decision or API/DB expansion. Technical details submitted for FN review: pure integer arithmetic, projected query shapes, atomic predicates, commit phases and bounded cleanup. C# methods, SQL lock behavior, race tests and telemetry integration remain unimplemented/unrun. Full gate awaits SPD and step 11. Submit 006_DD-FN with EN/JA PDFs; wait for approval before writing 006_DD-SPD. Approved inputs unchanged; no migration/code/external action in this phase.